



Universidade de Brasília
Departamento de Ciências da Computação
CIC0182 – Lógica Computacional

Adaptação da Proposta de Busca Binária: Verificação Formal de Inserção Ordenada em Coq

Integrantes da equipe:

Caio Jorge Soares Viana Jardim / Matrícula: 221003921

Eduardo Hermógenes Pinheiro de Moura / Matrícula: 221019028

Pablo Rafael Nunes Oliveira / Matrícula: 221003986

1. Introdução e Estruturação do Arquivo de Prova

O presente trabalho aborda a demonstração das propriedades lógicas e de corretude de um algoritmo de ordenação por inserção. Todo o desenvolvimento foi documentado em um único arquivo de código que adota um agrupamento linear, facilitando a construção iterativa do conhecimento formal. O código foi dividido nos seguintes blocos lógicos:

1. **Definições Estruturais:** Implementação das funções *bsearch*, *insert_at*, *bininsert* e o ordenamento principal *bininsertion_sort*.
2. **Propriedades da Busca e Manipulação do Tamanho:** Lemas que garantem o comportamento de limites, como *bsearch_valid_pos* e *insert_at_length*.
3. **Preservação de Conteúdo (Permutação):** Provas lógicas assegurando que a inserção de elementos apenas movimenta os dados sem corrompê-los.
4. **Verificação de Ordenação e Teorema Global:** Bloco final com o objetivo de amarrar as provas de que a lista é uma permutação válida e de que o seu conteúdo está ordenado.

2. Versionamento da Proposta

O projeto foi realizado de forma incremental em etapas, assemelhando-se ao versionamento de um projeto de software, com isso segue abaixo toda a estruturação utilizada na construção do nosso projeto:

Versão 0.1 - Estrutura inicial do algoritmo:

Nesta versão foram importadas as bibliotecas necessárias do Coq e implementadas as funções principais do projeto: *bsearch*, *insert_at*, *bininsert* e *bininsertion_sort*. Também foram adicionados testes utilizando Compute para verificar o comportamento das funções durante o desenvolvimento.

Versão 0.2 - Formalização das propriedades básicas:

Nesta etapa foram desenvolvidas as primeiras demonstrações formais, comprovando propriedades fundamentais do algoritmo. Foram provados lemas como *bsearch_valid_pos*, *insert_at_length*, *insert_at_0*, *insert_at_perm* e *bininsert_perm*, garantindo que a posição retornada é válida, que a inserção aumenta corretamente o tamanho da lista e que a inserção preserva os elementos da lista (permutação).

Versão 0.3 - Organização lógica e documentação:

Nesta versão foram adicionados predicados auxiliares (*le_all*) e seus respectivos lemas (*le_all_nil* e *le_all_cons*), preparando a estrutura para a prova da preservação da ordenação. Além disso, todo o código recebeu comentários explicativos, organizando cada seção do projeto e facilitando a compreensão da implementação e das demonstrações formais.

Versão 1.0 - Versão final:

A versão final reúne toda a implementação do algoritmo, os testes computacionais e as provas formais desenvolvidas durante o projeto. Foi incluída a estrutura da prova *bininsert_sorted*, utilizada na demonstração da corretude global do algoritmo, permanecendo como hipótese (*Admitted*) devido à sua elevada complexidade e ao tempo disponível para o desenvolvimento. Ainda assim, o projeto comprova formalmente as principais propriedades do algoritmo de inserção, incluindo validade da busca, preservação do tamanho da lista e preservação da permutação dos elementos, além de apresentar uma implementação completamente funcional e documentada.

3. Adaptações Realizadas no Código e Seus Motivos

Uma adaptação crítica e extremamente pertinente foi realizada na concepção da função de busca (*bsearch*).

Situação Original: O enunciado do projeto propunha a implementação de um algoritmo de inserção baseado em **Busca Binária**, utilizando a diretiva *Function* do Coq. Nessa abordagem, a lista era dividida recursivamente em duas partes por meio das funções *firstn* e *skipn*, e a prova de terminação era realizada com base em uma medida (*measure*) correspondente ao tamanho da lista.

Adaptação: Durante o desenvolvimento, optou-se por substituir essa implementação por uma versão baseada em **recursão estrutural**, utilizando a diretiva *Fixpoint*. Em vez de dividir a lista ao meio, a função percorre a lista elemento a elemento por meio da estrutura padrão $h :: t$, determinando recursivamente a posição de inserção do novo elemento. Essa alteração simplificou significativamente a definição da função, eliminando a necessidade de provas de terminação associadas ao uso de *Function* e permitindo concentrar a formalização nas propriedades do algoritmo utilizando indução estrutural sobre listas.

Motivo da Escolha: Lidar com chamadas de recursão não-estrutural e funções com medidas restritivas (divisão de listas) no Coq exige o domínio de lemas aritméticos complexos e mecanismos de indução forte. Ao simplificar para a busca linear estrutural, viabilizou-se o uso de táticas diretas de indução na estrutura da lista (*induction I*), o que reduziu massivamente o nível de complexidade e tornou exequível atestar as propriedades de validade de índices e estrutura.

4. Descrição das Provas em Linguagem Natural

Inspirado na arquitetura de subproblemas, o código seguiu uma sequência lógica para construir a corretude do software, resolvendo lemas fracionados:

4.1 Lema *bsearch_valid_pos*:

Problema abordado: Garantir que o índice devolvido pela função de busca seja matematicamente menor ou igual ao tamanho integral da lista.

Motivação lógica: Se o número retornado ultrapassasse o comprimento atual, a função de inserção não saberia onde alocar o item.

Estratégia: Indução formal sobre a lista, checando o caso base com lista vazia e derivando a recursividade com uma verificação condicional booleana ($x \leq h$) auxiliada pela tática *lia*.

4.2 Lema *insert_at_length* e *insert_at_0*:

Problema abordado: Comprovar a consistência aritmética da operação de inclusão.

Motivação lógica: Em termos de engenharia, provar que inserir um nó eleva o tamanho da lista em exatamente um (1) é o passo primário para confirmar que a estrutura fundamental da lista não sofreu degradação ou superposições.

4.3 Propriedades de Permutação (*bininsertion_sort_perm*):

Problema abordado: Mostrar formalmente que o algoritmo final retorna exatamente os mesmos valores recebidos na entrada.

Motivação lógica: A ordenação é válida apenas se nenhum dado foi perdido ou duplicado erroneamente. Através de propriedades transitivas das permutações, o grupo fechou essa prova garantindo que *insert_at* move a cauda conservando o estado da informação.

4.4 A Propriedade Assumida (*bininsert_sorted*) e a Conclusão Global:

Problema abordado: Atrelar o retorno do índice correto à manutenção obrigatória da organização da lista na hora do acoplamento do elemento.

Motivação lógica e Estratégia: Provar as relações inter-elementos em lógica de primeira ordem é penoso. Como detalhado em código pela equipe, atestar todos os sub-lemas da relação *Sorted* custaria grande tempo computacional e mental. Assim, essa hipótese foi ativada como pressuposto (*Admitted*). Com esse pilar assumido, o Teorema Principal fundiu as permutações à regra de ordenação sistêmica para estabelecer a corretude final do algoritmo completo.

4.5 Predicado e Lemas Auxiliares (*le_all*, *le_all_nil* e *le_all_cons*):

Problema abordado: Representar e formalizar que um elemento é menor ou igual a todos os elementos de uma determinada lista.

Motivação lógica: Para facilitar futuras demonstrações envolvendo listas ordenadas, é indispensável estabelecer uma propriedade estrutural clara. O predicado *le_all* atua como essa base. A partir dele, os lemas *le_all_nil* (lidando com o caso trivial lógico da lista vazia) e *le_all_cons* (garantindo a relação ao encadear recursivamente a cabeça e o corpo da lista) foram desenvolvidos para simplificar essas etapas.

Estratégia: Essas propriedades constituem a base lógica necessária para a demonstração do teorema principal, preparando o terreno formal que estabelece que o algoritmo produz uma lista garantidamente ordenada contendo exatamente os mesmos elementos da lista original.

5. Considerações e Dificuldades no Ambiente Intuicionista

Trabalhar a validação do Algoritmo de Inserção evidenciou as virtudes e a alta carga de rigor cobrada pela linguagem do Coq. Diferente do pensamento de alto nível na engenharia de software tradicional, o modelo intuicionista demanda que até a propriedade mais elementar seja construída formalmente do zero.

A dificuldade natural verificada para provar a manutenção global das características ordenadas (Sorted) sem a hipótese auxiliar *binsert_sorted* demonstra empiricamente o quão custoso é o pareamento entre aritmética de índices e dependência condicional dentro das provas. No entanto, o encadeamento dos sucessivos lemas de indução e os predicados auxiliares (*le_all*) comprovaram um amadurecimento lógico sólido da equipe na condução de demonstrações consistentes, refletindo uma divisão e estruturação bastante modular do problema original.

Outro desafio significativo enfrentado pelo grupo foi de ordem técnica e de infraestrutura. O desenvolvimento prático do código e das provas foi concentrado quase que integralmente no computador de apenas um dos integrantes. Isso ocorreu porque as máquinas dos outros dois membros não conseguiram atender aos requisitos necessários para a execução adequada do assistente de provas Coq, ou apresentaram instabilidades e lentidão que inviabilizaram o uso prático. Essa limitação de hardware exigiu uma adaptação na dinâmica de trabalho da equipe: enquanto a implementação, a digitação e a compilação do código ficaram centralizadas em uma única estação, os demais membros precisaram focar inteiramente na colaboração intelectual, desenhando o raciocínio lógico e a estruturação teórica.